

Outputs and Workbook Reference

How Results Are Organized

OxCidation exposes the same experiment at three levels:

Level	Best source	Use it to answer
Experiment	<code>summary.json</code> and <code>workbook Overview</code>	Which prompt performed better overall?
Program and prompt	<code>results.csv</code> , <code>Automatic Results</code> , and <code>Review Queue</code>	What happened to this translation from initial to final code?
Attempt and evidence	<code>Attempts</code> , <code>Artifact Index</code> , and <code>agent_interactions.json</code>	Why did it fail or get repaired?

This separation keeps the spreadsheet readable without discarding detailed research evidence. The CSV and JSON files remain the machine-readable source of truth; the workbook is the main navigation and inspection surface.

A Practical Reading Path

To compare prompts

1. Open `Overview` for aggregate initial acceptance, final acceptance, and repair counts.
2. Use `Automatic Results` to filter by prompt, failure type, or Judge verdict.
3. Check problem and Judge coverage before interpreting percentages.

To investigate one result

1. Find the row in `Review Queue`.
2. Use its `Review ID` to locate the same result in other sheets.
3. Open code and reports through `Artifact Index`.
4. Read `Attempts` for the compact repair history.
5. Open `agent_interactions.json` when the complete causal sequence is needed.

To distinguish prompt quality from repair quality

Compare the initial compilation, visible-test, and Judge fields with their final counterparts. The initial Rust file is preserved separately from the final `translated.rs`.

Workbook Scope

`review_workbook.xlsx` contains curated automatic fields and relative links. Links display `Open` and remain valid when the complete experiment folder is shared, without embedding absolute paths from the machine that ran it.

The workbook intentionally contains no reviewer-entry, blinding, scoring, or adjudication forms. It is an automatic analysis and artifact-navigation file.

Overview

The top section identifies the run:

Field	Meaning
Experiment	Unique experiment identifier.
Model provider	Configured LLM provider.
Model	Exact configured model identifier.
Programs	Number of selected C programs.
Prompts	Number of translation prompts.
Completed combinations	Completed program-prompt results.

The second section summarizes each prompt:

Column	Meaning
Prompt	Prompt identifier.
Runs	Completed combinations for the prompt.
Initial accepted	Initial Rust translations accepted by the configured Judge.
Final accepted	Final Rust translations accepted by the configured Judge.
Repaired	Results with at least one repair.
Judge failures	Final Judge verdicts other than ACCEPTED, SKIPPED, or not_reached.
Judge skipped	Final Judge stage was reached, but evaluation was not performed, such as when the C baseline was not valid.
Judge not reached	Final Judge stage was not executed; expected to remain zero in complete runs.

`SKIPPED` is neither a pass nor a translation failure. Its reason must be examined to determine why the combination was not evaluable by the local Judge.

Review Queue

This sheet is a compact list of all program-prompt results. It is useful for finding failures without reading the larger automatic table.

Column	Meaning
Review ID	Stable identifier shared by workbook sheets.
Snippet ID	CodeNet submission identifier.
Problem ID	CodeNet problem identifier.
Prompt ID	Translation prompt identifier.

Visible Suite Status	Whether the shared generated suite is ready or unavailable.
Visible Suite C Compile Profile	Local C profile that produced the visible-test oracle, such as gnu11 or gnu89-compat. Empty when neither profile compiled.
Initial Compile Status	Compilation result of the initial Rust code.
Initial Visible Test Status	Initial visible-suite result.
Initial Judge Status	Judge result for initial Rust.
Final Status	Terminal pipeline status.
Final Judge Status	Judge result for final Rust.
Repair Attempts	Number of generated repairs.
Failure Category	Terminal pipeline failure category.
Visible Failure Example	One compact visible-test failure.
Judge Failure Example	First final Judge failure.

Automatic Results

This sheet contains the principal automatic measurements, one row per C program and translation prompt.

Column	Meaning
Review ID	Stable identifier shared by workbook sheets.
Snippet ID	CodeNet submission identifier.
Problem ID	CodeNet problem identifier.
Prompt ID	Translation prompt identifier.
Initial Compile Status	Initial Rust compilation outcome.
Initial Visible Test Status	Initial visible-suite outcome.
Initial Judge Status	Initial Rust Judge verdict.
Final Status	Terminal pipeline status.
Failure Category	Terminal pipeline failure class.
Visible Test Status	Final visible-suite outcome.
Visible Suite Status	Whether preparation produced a frozen usable suite. baseline_compile_failed clearly excludes the visible suite from translation evidence.

Visible Suite Review Status	Pre-translation review result for the shared visible suite.
Visible Suite C Compile Profile	Successful local C compatibility profile. Empty means every configured profile failed.
Visible Suite Review Count	Number of complete-batch Validator reviews attempted, up to two.
Visible Suite Generation Attempts	Number of initial or replacement generation calls, up to three.
Visible Suite Deterministic Rejections	Generated candidates rejected because they were duplicate, failed, or timed out when executed on C.
Visible Suite Validator Replacements	Cases the Validator requested to replace across batch reviews.
Visible Suite Unresolved Replacements	Requested slots that could not be filled within the bounded preparation flow.
Visible Cases Total	Generated visible cases executed.
Visible Cases Passed	Visible cases passed by final Rust.
Judge Status	Final Rust Judge verdict.
Judge Cases Passed	Judge cases accepted by final Rust.
Judge Cases Failed	Judge cases failed by final Rust.
Judge First Failure Verdict	Verdict of the first failed Judge case.
Repair Attempts	Number of generated repairs.
Duration Seconds	Total recorded pipeline duration.

Attempts

Each row represents one Rust version: attempt `0` is the initial translation and later rows are repairs.

Column	Meaning
Review ID	Parent result identifier.
Snippet ID	CodeNet submission identifier.
Problem ID	CodeNet problem identifier.
Prompt ID	Translation prompt identifier.
Attempt Number	<code>0</code> for initial code; repairs start at 1.
Attempt Kind	initial or repair.
Compile Status	Compilation outcome for this code version.

Compiler Output Excerpt	Compact compiler evidence.
Visible Test Status	Visible-suite outcome for this code version.
Visible Cases Total	Visible cases executed.
Visible Cases Passed	Visible cases passed.
Visible Cases Failed	Visible cases failed.
Visible Verdict Counts	Per-verdict visible-test counts.
Validator Test Assessment	Translation discrepancy, invalid visible test, or no required analysis.
Validator Diagnosis	Concise semantic diagnosis.
Validator Repair Guidance	Guidance supplied to the next repair.
Repair Failure Category	Previous failure that triggered this repair, not this attempt's result.
Initial Judge Status	Initial-code Judge verdict, normally on attempt 0.
Final Judge Status	Final-code Judge verdict, normally on the final attempt.

The repair failure category is easy to misread. It explains why an attempt was created. To know whether that attempt succeeded, use its compile and visible- test status.

Artifact Index

This sheet centralizes links instead of displaying raw file paths throughout the workbook.

Column	Opens or identifies
Review ID	Stable result identifier.
Snippet ID	CodeNet submission identifier.
Problem ID	CodeNet problem identifier.
Source	Original C source.
Raw Model Output	Raw initial model response.
Initial Translation	Initial Rust code.
Final Translation	Final Rust code.
Visible Test Report	Final visible-test report snapshot.
Visible Test Suite	Generated suite manifest and case reference.
Visible Suite Preparation	Shared generation, deterministic validation, and review history.
Validator Report	Final Validator report snapshot.
Agent Interactions	Complete chronological interaction history.

Attempt Summary	Structured list of all code attempts.
Initial Judge Report	Initial C/Rust Judge comparison.
Final Judge Report	Final C/Rust Judge comparison.
Complete Result	Full result JSON.

Files Produced by an Experiment

Every run is isolated under `outputs/<experiment-id>/` :

File or directory	Purpose
<code>manifest.json</code>	Frozen sources, prompts, model, seed, implementation identity, and Judge profile.
<code>results.csv</code>	Complete result table, one row per program-prompt combination.
<code>attempts.csv</code>	Complete attempt table, one row per Rust version.
<code>summary.json</code>	Aggregate outcomes by prompt.
<code>review_workbook.xlsx</code>	Human-readable analysis and artifact navigation.
<code>runs.jsonl</code>	Incremental structured workflow records.
<code>resume_history.jsonl</code>	Accepted resume operations.
<code>logs/</code>	Text logs isolated to this experiment.
<code>visible_tests/</code>	Generated suites shared across prompts.
<code>model_outputs/</code>	Per-program and per-prompt code and reports.

Inside `model_outputs/<submission>/<prompt>/` :

Artifact	Meaning
<code>source.c</code>	Original accepted C program.
<code>raw_response.txt</code>	Raw initial model response.
<code>attempts/attempt-00-initial.rs</code>	Initial Rust translation.
<code>attempts/attempt-NN-repair.rs</code>	Versioned repaired Rust code.
<code>translated.rs</code>	Final Rust code.
<code>attempts.json</code>	Structured attempt summary.
<code>test_report.json</code>	Final visible-test report snapshot.
<code>validator_report.json</code>	Final Validator report snapshot.
<code>agent_interactions.json</code>	Complete chronological agent and tool evidence.

<code>initial_judge_comparison.json</code>	C and initial Rust Judge comparison.
<code>judge_comparison.json</code>	C and final Rust Judge comparison.
<code>result.json</code>	Complete machine-readable result.

`test_report.json` and `validator_report.json` are convenient final snapshots. Use `attempts.json` and `agent_interactions.json` when the complete history is needed.

The full `results.csv` and `summary.json` contain generated candidate and frozen case counts in addition to the preparation fields shown in the workbook. Detailed inputs, verdicts, reasons, and replacement requirements remain in the shared suite preparation history instead of being repeated for every prompt.